CS 165 COMPUTER SECURITY • UC Riverside • Fall 2026 • **MIDTERM**

Instructions

- Closed book. No notes, calculators, phones, or other devices.
- **Six questions, each worth 20 points. Answer any FIVE**, for a maximum of 100.
- Tick the five you want marked in the box below. If you tick more than five, the first five answers will be considered.
- Answer in the space provided; continue on the back of the page if you need to.
- Short answers are expected — a few sentences, or a labeled sketch, is enough.
- Points are shown for each part and are a good guide to how much to write.
- Write as clearly as possible; if I can't understand your writing, you will lose points.
- Always (concisely) justify your answers.

Mark these five questions tick up to five

☐ **1** ☐ **2** ☐ **3** ☐ **4** ☐ **5** ☐ **6**

If you tick nothing, the first five questions you attempt will be marked.

1 Threat models and attack surface — 20 points

A public server runs `photod`, a thumbnail service that (1) reads images uploaded by any user, (2) reads a watermark file from a path only `root` can write, and (3) writes finished thumbnails into a web directory. Its image parser contains a known out-of-bounds write.

(a) [3 pts] Name the three properties of the **CIA triad**.

Answer. Confidentiality, **Integrity**, **Availability**. *1 each.*

(b) [7 pts] Recall that a vulnerability is a flaw that is *accessible* to an adversary who can *exploit* it. For an adversary on the Internet, is the parser bug a vulnerability when reached by the path in (1)? What about when reached by path (2)? Justify each answer in terms of accessibility.

Answer. Path (1): yes, a vulnerability. Any Internet user can upload an image, so the adversary directly supplies the input that drives the flawed parser — the flaw is accessible, and an out-of-bounds write can be used to violate security. **Path (2): no, not for this adversary.** The same flaw is there, but only `root` can write the watermark file, so an Internet adversary cannot supply the input that triggers it; it is not accessible, so it is a flaw but not a vulnerability under this threat model. *3 points per path for the correct verdict with an accessibility-based justification, 1 for noting it is the same flaw and only the threat model differs, and only the threat model differs.*

(c) [4 pts] Recall that the *attack surface* is the set of system calls a program makes that access adversary-modifiable resources. Which of (1)–(3) are on it here?

Answer. Only **(1)** is on the attack surface: the uploaded image is a resource the adversary can modify. (2) is off it because the watermark file is root-only, and (3) is off it because the thumbnail directory is not adversary-modifiable in this threat model — accept either justification for (3): that the thumbnail directory is not an adversary-modifiable resource, or that the call receives no adversary input (Lecture 2 states the test both ways). *2 for identifying (1) alone, 2 for the justification.*

(d) [6 pts] The Morris worm (1988) hijacked `fingerd` through a buffer overflow and copied itself from host to host, reinfecting machines many times over. It stole no data and modified none. **(i)** Which of confidentiality, integrity and availability did it mainly violate, and why? **(ii)** What changed in commercial systems as a result?

Answer. (i) Mainly **availability**: the worm stole and altered no data, but repeatedly reinfecting the same machines consumed their resources until they ground to a halt — an Internet-scale denial of service. Credit also for noting it violated the **integrity of the `fingerd` process**, which ran the worm's code instead of its own, and that password cracking brushes against confidentiality. (ii) Strikingly little: there were **few tangible security changes in commercial systems**. Morris was punished and blame was assigned to him personally; the one real exception was a push in **network security research**, and systems went on to take *more* risks (for example executable email attachments). *3 for availability with the denial-of-service reasoning, 3 for "almost nothing changed" — accept the research exception or the increased risk-taking as the detail.*

2 Stack overflow — 20 points

This 32-bit function is compiled **without** a stack protector, and `req` and `len` are attacker-controlled:

```
void handle(char *req, int len) {
    int authed = 0;
    char buf[64];
    memcpy(buf, req, len);
    if (authed) grant_access();
}
```

Assume the compiler places `authed` immediately above `buf`, then the saved frame pointer (4 bytes), then the saved return address (4 bytes).

(a) [3 pts] In one short sentence, what is the main idea of a **buffer overflow**?

Answer. Writing (or reading) past the end of a buffer, into memory that belongs to something else. *3; accept any answer conveying "access outside the bounds of the object".*

(b) [3 pts] In which direction does `memcpy` write relative to `buf`, and in what order does a growing overflow reach the other three items?

Answer. The stack grows toward *low* addresses, but a copy into `buf` writes toward *high* addresses. So a growing overflow reaches `authed` first, then the saved frame pointer, then the saved return address. *2 for the write direction (and that it is the opposite of stack growth), 1 for the order.*

(c) [5 pts] Show your arithmetic for each: **(i)** how many bytes of filler precede the saved return address, and **(ii)** the smallest `len` that overwrites its first byte.

Answer. (i) 64 (`buf`) $+ 4$ (`authed`) $+ 4$ (saved frame pointer) $=$ **72** bytes of filler. (ii) The 73rd byte is the first byte of the saved return address, so the smallest `len` is **73**. *2 for (i), 3 for (ii); full credit either way if the arithmetic is shown and the arithmetic shown.*

(d) [6 pts] The attacker chooses how many bytes to send. Describe two *different* attacks this one bug allows, and say what the attacker achieves in each. Does DEP/NX prevent the *overwrite*? If it does, then how? If it does not, then why not?

Answer. Short overflow: reach `authed` and set it nonzero, so `grant_access()` runs although authentication failed — a *data-only* attack with no control-flow hijack at all. **Long overflow:** reach and overwrite the saved return address, so the attacker chooses where the function returns — a full control-flow hijack. **DEP/NX does *not* prevent the overwrite:** NX governs whether a page may be *executed*, not whether it may be written, so the overwrite and the hijacked `ret` both still happen; NX only blocks the classic follow-on of executing injected shellcode, and code reuse gets around that anyway. *2 per attack, 2 for "no" with the execute-vs-write reason.*

(e) [3 pts] Recall that memory errors are classified as *spatial*, *temporal* or *type*. Which kind is the bug in `handle()`, and why? Give one example each of the other two kinds.

Answer. The bug in `handle()` is a **spatial** error: `memcpy` writes outside the bounds of `buf`. **Temporal** — the object is used at the wrong time: a use-after-free (writing through a pointer after `free()`), or a use-before-initialization (reading a local never assigned to). **Type** — the same bytes read through the wrong type: a `union` field written as a float and read as a pointer, or a bad cast. *1 for spatial with the reason, 1 for each valid other example.*

3 Return-oriented programming — 20 points

NX stops shellcode, and often no single library function does what you want. ROP chains short sequences ("gadgets") that each end in `ret`. Suppose you have found:

```
g1:  pop eax ; ret
g2:  pop ebx ; ret
g3:  mov [ebx], eax ; ret
```

(a) [4 pts] What does **ROP** stand for, and what instruction does every gadget end in?

Answer. Return-Oriented Programming; every gadget ends in `ret`. *2 + 2.*

(b) [8 pts] Lay out the stack (the sequence of values the overflow writes) to store value `V` at address `ADDR` using these gadgets, and say what each `ret` does.

Answer. Lay the stack as: `[&g1] [V] [&g2] [ADDR] [&g3]`. On the first `ret` control enters `g1`: `pop eax` takes `V` off the stack into `eax`, and `g1`'s `ret` pops `&g2`. `g2`: `pop ebx` takes `ADDR` into `ebx`, `ret` pops `&g3`. `g3`: `mov [ebx], eax` writes `V` to `ADDR`, `ret` continues to whatever gadget is next. The stack pointer walks down the chain — the attacker-controlled stack *is* the program, each `ret` advancing to the next instruction. *5 for the correct five-slot layout in the right order, 3 for explaining the `ret`-driven execution.*

(c) [3 pts] Why does NX not stop ROP?

Answer. Every gadget is existing, already-executable code in the binary or libc; nothing writable is executed. NX only blocks running injected data, so reusing code sails past it. *3.*

(d) [5 pts] What does ASLR force the attacker to do first?

Answer. Gadget addresses depend on where code is mapped, which ASLR randomizes, so `&g1`, `&g2` and `&g3` are not known in advance. The attacker must first obtain an information leak that discloses a code address, from which the module base and therefore every gadget address follows (or find a module that is not randomized). *2 for "the addresses are unknown / move every run", 3 for needing a leak to recover the base.*

Note. Reused verbatim from Exercises 2, Q6 — stem, gadget listing and all three parts unchanged. It is the only question reproduced word for word, but note that much of the rest of the paper is close (not verbatim) reuse of Exercises 2 as well, so it is not a clean control. Original wording from the exercises; it is here deliberately, to reward students who worked the sheet.

4 Shellcode — 20 points

Instead of reusing existing code, the attacker writes their own machine instructions into the buffer and returns into them.

(a) [4 pts] What is **shellcode**?

Answer. A short sequence of machine instructions the attacker gets into the process as data and then executes; the classic payload spawns a shell (`/bin/sh`). *4; 2 if they only say "the attacker's code".*

(b) [6 pts] The same bytes are written as data and then executed as code. Explain how that is possible, and state the one assumption about the stack it depends on.

Answer. Nothing in memory marks bytes as "data" or "code" — the copy writes them as data, and after the overwritten `ret` transfers control into the buffer the CPU simply fetches and executes them as instructions. This is the same code/data confusion introduced in Lecture 1. It depends on the **stack page being executable**; with DEP/NX enabled it fails, which is exactly why Lecture 8 moves to code reuse. *4 for the code/data explanation, 2 for naming the executable-stack assumption.*

(c) [5 pts] Shellcode delivered through a string copy such as `strcpy` must contain no NUL (`0x00`) bytes. Why? Give the standard trick for zeroing a register without introducing one, and name one delivery path where the constraint does *not* apply.

Answer. Because the overflow is typically delivered through a string copy such as `strcpy`, which stops at the first NUL — a zero byte inside the payload would truncate it, so the rest of the shellcode would never be written. The standard trick is to zero a register with `xor reg, reg` (e.g. `xor eax, eax`) rather than loading an immediate `0`, since the `xor` encoding contains no zero byte. The constraint is specific to NUL-terminated copies: a length-driven delivery such as `memcpy` (as in Question 2) or a `read()` of `len` bytes copies zero bytes happily, so shellcode sent that way may contain NULs. *2 for the string-copy truncation reason, 2 for `xor reg,reg`, 1 for a length-driven delivery path.*

(d) [5 pts] The classic payload makes a single system call. Which one, and how are the call number and its arguments passed on 32-bit Linux?

Answer. `execve("/bin/sh", NULL, NULL)`, which replaces the running process image with a shell. On 32-bit Linux the system-call number goes in `eax` and the arguments in `ebx`, `ecx`, `edx`, then the call is made with `int 0x80`. *2 for execve spawning a shell, 2 for `eax` + `ebx/ecx/edx` + `int 0x80`.*

5 Attacking the heap — 20 points

(a) [4 pts] What does `free()` do, and what is a **dangling pointer**?

Answer. `free()` returns a heap allocation to the allocator so the space can be reused. A dangling pointer still points at memory that has already been freed. *2 + 2.*

(b) [9 pts] An attacker overflows the chunk before chunk `P` and so controls `P`'s `fd` (forward pointer) and `bk` (backward pointer). (i) What does the unlink performed by `free(P)` hand them? (ii) Name *two distinct* targets they could overwrite with it, and say what each one gives them. (iii) State the check "safe unlinking" adds, and why forged pointers fail it.

Answer. (i) A **write-what-where** — an arbitrary write of a chosen value to a chosen address, because the splice writes using the attacker's `fd` (value) and `bk` (target, plus a fixed offset, as on the Lecture 7 slide). (ii) Any two of: a **function pointer** in a live object (Lecture 7) — the next indirect call through it is redirected; a **saved return address** on the stack (Lecture 5) — the next `ret` goes to the attacker's address. Also accept a **GOT entry**, though the GOT is not covered in Lectures 1–10, so do not expect it. (iii) Safe unlinking asserts the list is consistent around `P`: `P->fd->bk == P` and `P->bk->fd == P`. Forged `fd`/`bk` cannot satisfy both at once, so `free()` detects the tampering and aborts. *3 for write-what-where, 3 for two targets with their effects, 3 for the invariant and why it fails.*

(c) [7 pts] A use-after-free leaves a stale pointer `b`. Explain why such a bug is often *not* exploitable as it stands, and what the attacker does to make it reliable.

Answer. If nothing has reused the freed slot, a write through `b` merely scribbles on free memory — there is no victim, so the bug is inert. It becomes exploitable once the allocator hands that slot to an object the attacker cares about, so the write corrupts that object (a write primitive). Since they cannot place objects directly, the attacker **grooms** (sprays) the heap: issuing many allocations of the right size and type so one reliably reuses the freed slot. *3 inert, 4 grooming/spraying for reliable reuse.*

6 Defenses — 20 points

(a) [3 pts] Name the three *classic* defenses covered in this course.

Answer. Stack canaries, **ASLR**, **DEP/NX**. *1 each.*

(b) [4 pts] Give *one* way a stack canary is bypassed.

Answer. Leak it and write it back unchanged so the pre-`ret` check still passes; or corrupt a target a contiguous overflow need not cross the canary to reach (a local pointer, a flag such as `authed`, heap metadata). *4 for either route, clearly justified. Ignore: 1.5, 2.*

(c) [7 pts] Recall that control hijacking has *two targets*. Name each, name the defense for it, and give one line on how that defense works.

Answer. The saved return address — the backward edge, protected by a **shadow stack**: each call copies the return address into a separate protected region, and on return the two are compared, aborting on a mismatch (Intel CET implements this in hardware). **Function pointers / indirect calls — the forward edge**, protected by **CFI**: each indirect transfer is restricted to the targets the program's code legally allows, so a pointer overwritten to aim at a gadget falls outside the set and the call is refused. *1.5 per target, 2 per defense with its mechanism.*

(d) [6 pts] Intel CET enforces only *coarse* CFI on the forward edge. Explain why an attacker can still find a usable target inside the coarse allowed set, and state what fine CFI would additionally check. Does that guarantee the target is excluded?

Answer. Coarse CFI permits an indirect call to land on *any* function entry point, so the allowed set is huge — it still contains functions the attacker finds useful (for example `system`, `execve`, or any entry point that advances their chain). The transfer is therefore "legal" under the policy even though it is not one the program would ever have made. **Fine** CFI narrows the set by requiring the target to match the *signature* of the function pointer being called — the return type and the number and types of the arguments — so a call through, say, a `void (*)(void)` pointer can no longer land on `system(char *)`. **No, it does not guarantee exclusion:** a gadget whose signature happens to match still passes the check, and a data-only attack never leaves the legal set at all — which is why Lecture 9 titles the next slide "CFI Is Not a Complete Fix". *2 for "any function entry is allowed, so useful ones remain in the set", 2 for signature matching, 2 for answering "no" with a reason (same-signature gadget, or data-only attack).*